Couchbase SDKs Overview

Fast, Flexible Access to Data at Scale

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Agenda

1. Couchbase Programming Model
2. Overview of Operations
3. Transactions
4. Asynchronous API
5. SDK Field Level Encryption
6. Error Handling
7. Observability

Programming Model

Couchbase SDKs Benefits

Ease of development

- Native support in programming languages
- Discoverable API via IDE
- Robust SDK supporting connection management, error handling, timeouts and retry

Feature rich

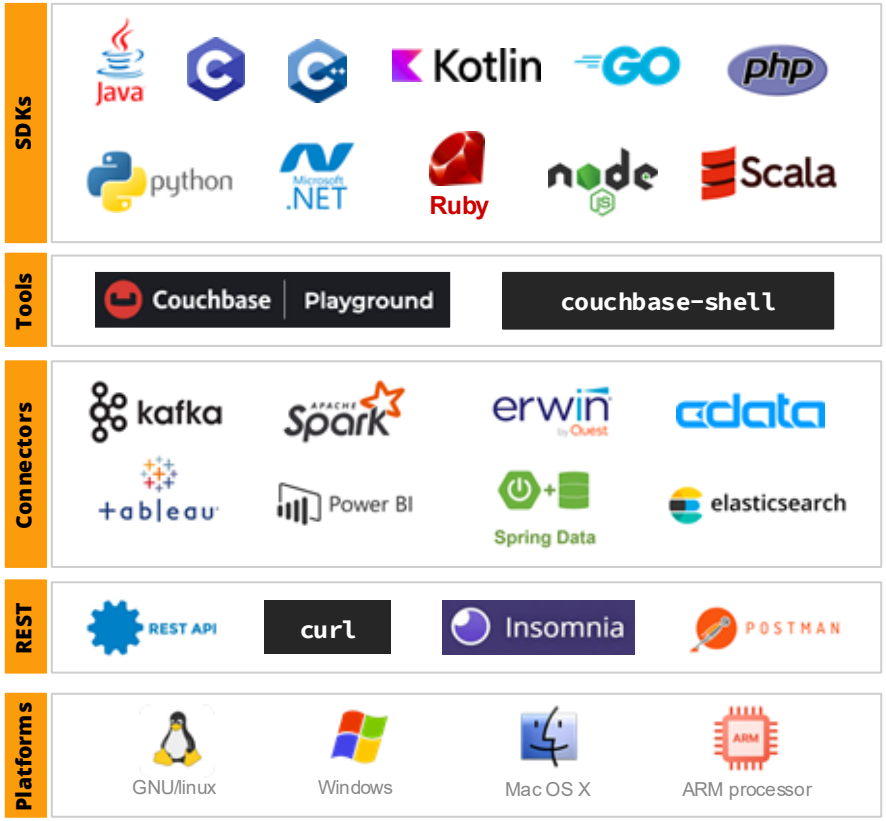
- Easily navigate Couchbase hierarchy
- Topology awareness
- Transactions support
- Sync and Async APIs
- Metrics & observability
- Security & encryption

Idiomatic

- Unified API across languages
- Consistent behavior
- Fluent and readable syntax
- Language specific design, when applicable.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Couchbase Developer Platform



Couchbase SDKs

- 11 Couchbase SDKs to choose from
- We strongly advise using the latest version of your preferred SDK

Couchbase Tools

- Couchbase Playground
- Couchbase CB Shell

Connectors

- Kafka, Spark, ElasticSearch
- BI connectors & tools

REST API

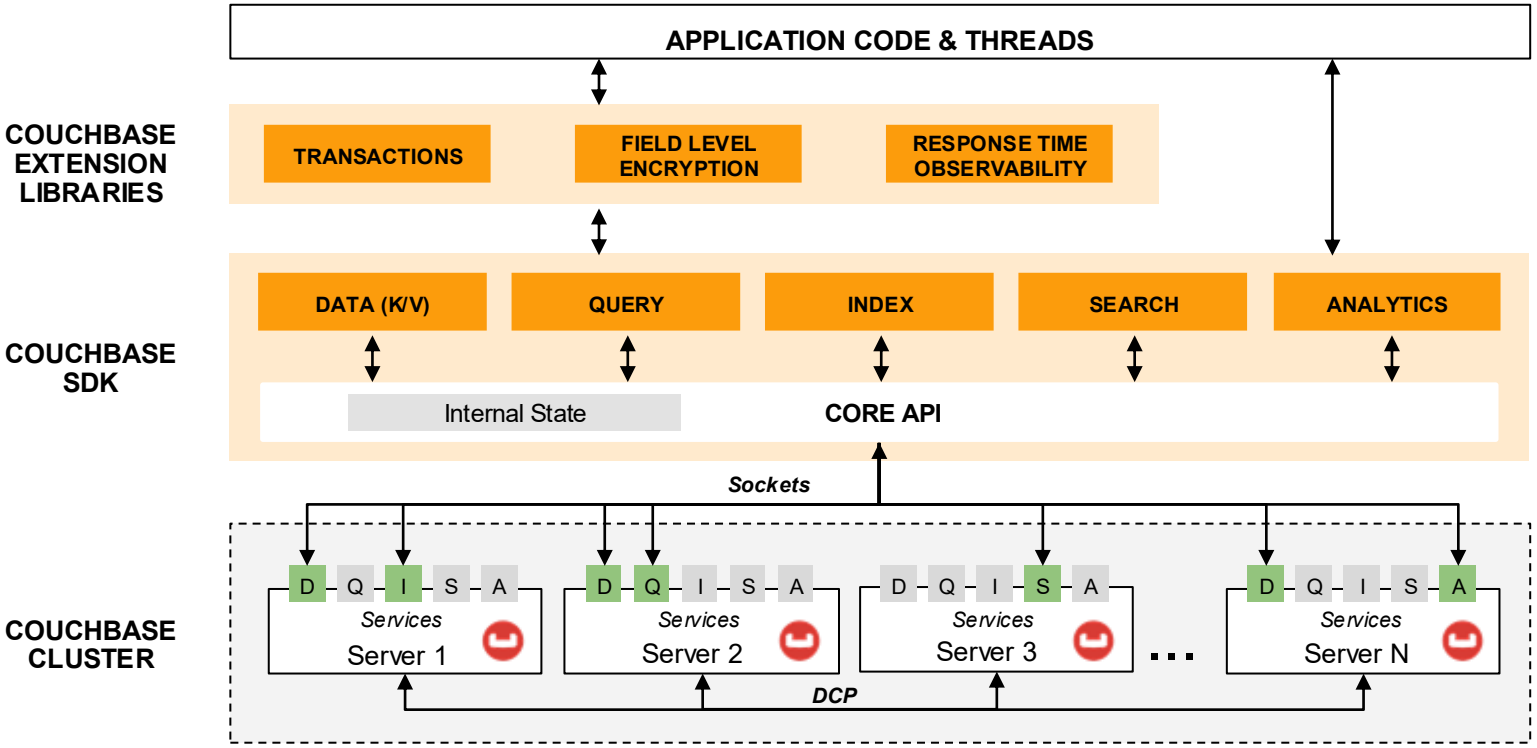
- CURL
- Postman, Insomnia, etc.

Platforms

- GNU/Linux, Microsoft Windows, Mac OS X
- ARM processors

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

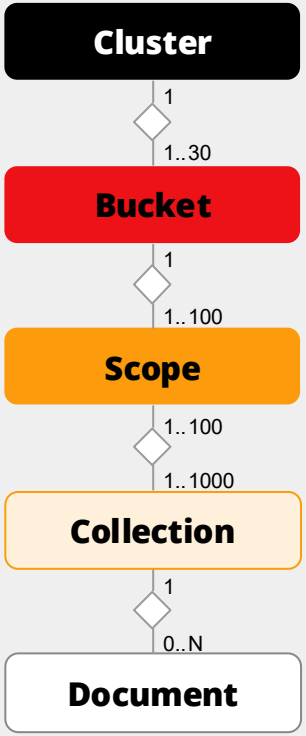
Intelligent Application SDK with Unified API



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SDK Common Entities

- All SDKs have a equivalent Class or variable type mapping to Couchbase database hierarchy
- These entities are created on the server side and represents the server side elements



- Provides access to cluster-level operations: SQL++, Analytics, Search
- Manages bootstrapping, fault tolerance, rebalancing

- Provides access to bucket level operations
- Manages persistence, replicas, TTL, compression mode, XDCR resolution, ejection policy, compaction

- Unique keyspace of Collections
- Applications can be assigned per-scope access-rights

- Unique keyspace of Documents
- Collections might be assigned to different scopes.

- JSON: numbers, strings, embedded documents and arrays
- Non JSON: UTF8, buffer, bytearray, serialized objects

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Goal of a Couchbase SDK

Ruby

- Abstract a programming model over a Couchbase cluster
- Adapt to changes in the cluster over time
- Mask transient failure conditions from application logic
- Connect high-level platform concepts to document database approaches

```
require "couchbase"
include Couchbase # to avoid repeating mod name

options = Cluster::ClusterOptions.new
options.authenticate("user", "password")
cluster = Cluster.connect("couchbase://localhost", options)

bucket = cluster.bucket("bucket-name")
collection = bucket.default_collection

upsert_result = collection.upsert(
      "u:avsej",
      {
                        "name" => "Sergey Avseyev",
                        "interests" => ["Ruby"]
      }
)
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SDK and Versioning

- Couchbase SDKs have an API version, implemented by specific library versions (e.g. “SDK 3.0”)
- All Couchbase SDK’s follow semantic versioning (Major.Minor.Patch)
- The minor versions may correlate with a Server release

Couchbase SDK Version	Couchbase Server Version
3.0	6.5
3.1	6.6
3.3	7.0
3.6	7.6

SDK Release Version to API Version map

	API 3.2	API 3.3	API 3.4	API 3.5	API 3.6
.NET	3.2	3.3	3.4	3.5	3.6
C	3.2	3.3.0-3.3.2	3.3.3	N/A	N/A
C++	-	-	-	-	1.0
Go	2.3 & 2.4	2.5	2.6 & 2.7	2.8	2.9
Java	3.2	3.3	3.4 & 3.5	3.6	3.7
Kotlin	-	1.0	1.1 & 1.2	1.3	1.4
Node.js	3.2 & 4.0	4.1	4.2	4.3	4.4
PHP	3.2	4.0	4.1	4.2	4.3
Python	3.2	4.0	4.1	4.2	4.3
Ruby	3.2	3.3	3.4	3.5	3.6
Scala	1.2	1.3	1.4 & 1.5	1.6	1.7

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Overview of Operations

Code examples in Java

Java | Connection Management

1. A connection to a Couchbase cluster is represented by a `Cluster` object.

   A `Cluster` provides access to buckets, scopes, and collections.

   The simplest way to create a `Cluster` object is to call `Cluster.connect()` with a connection string, username, and password.

1. SDK 3.4 introduced a `wan-development` Configuration Profile, which provides pre-configured timeout settings suitable for working in high latency environments.

1. Get a handle to `Bucket`, if desired.

```
var connectionString = "couchbases://...";
var username = "...";
var password = "...";

Cluster cluster = Cluster.connect(
        connectionString,
        ClusterOptions.clusterOptions(username,
password)
                    .environment(env -> {
          // Sets a pre-configured profile called
"wan-development"
          env.applyProfile("wan-development");
        })
    );

// Get a bucket reference
Bucket bucket = cluster.bucket(bucketName);
bucket.waitUntilReady(Duration.ofSeconds(10));
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Java | KV Get

1. Simply use the `collection.get("doc-key")` to fetch your documents.

1. Once we have a `GetResult`, we can use `contentAsObject()` to turn the content back into a `JsonObject` like we inserted it in the examples before, or use the more generic `contentAs(T.class)` equivalent to turn it back into other entity structures.

   In fact, the `contentAsObject()` method is just a convenience method for `contentAs(JsonObject.class)`.

```
// Get a user-defined collection reference
Scope scope = bucket.scope("tenant_agent_00");
Collection collection = scope.collection("users");

try {
  GetResult getResult = collection.get("doc-key");
  String title =
getResult.contentAsObject().getString("title");
  System.out.println(title); // title == "My Blog
Post"
} catch (DocumentNotFoundException ex) {
  System.out.println("Document not found!");
}
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Java | KV Upsert and Insert

1. Simply use the `collection.upsert("doc-key", content)` to insert the document if it does not exist, or replace it if it does.
2. Simply use the `collection.insert("doc-key", content)` to insert the document.
3. Insert works very similarly to upsert, but will fail if the document already exists with a `DocumentExistsException`

Note that we also support Asynchronous API - presented in a separate section

```
JsonObject content = JsonObject.create()
 .put("author", "mike")
 .put("title", "My Blog Post 1");

MutationResult result = collection.upsert("doc-key", content);
```

```
try {
 JsonObject content = JsonObject.create()
     .put("title", "My Blog Post 2");
MutationResult insertResult =
    collection.insert("document-key2", content);
} catch (DocumentExistsException ex) {
 System.err.println("The document already exists!");
} catch (CouchbaseException ex) {
 System.err.println("Something else happened: " + ex);
}
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Java | KV Replace & Remove

1. Use `collection.replace(...)` to replace the contents of a document. If we get a CAS mismatch, we usually just want to retry the operation.

1. Removing a document is straightforward using `collection.remove("doc-id")`

```
// Retry example which retries on CAS failure
while (true) {
 try {
   collection.replace(id, content,
     replaceOptions().cas(found.cas()));
   break; // if successful, break out of the retry loop
 } catch (CasMismatchException ex) {
   // don't do anything, we'll retry the loop
 }
}

try { // Remove example
    collection.remove("doc-id");
} catch (DocumentNotFoundException ex) {
    System.out.println("Document did not exist when trying to
remove");
}
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Java | Document expiration (TTL)

1. You can set an expiry value from a `Duration` when creating a document.

1. When getting a document from Couchbase Server, the expiry is not included by default, but it can be requested by setting the `withExpiry` option to true.

1. Note that when updating the document, special care must be taken to avoid resetting the expiry to zero. If you are using Couchbase Server 7.0 or later, set the `preserveExpiry` option when updating the document.

1. Some applications may find `getAndTouch` useful, which fetches a document while updating its expiry field.

```
1  MutationResult insertResult =
   collection.insert("my-document2", json,
   insertOptions().expiry(Duration.ofHours(2)));

2  // KV Get a document with expiry
   GetResult result = collection.get("my-document3",
   getOptions().withExpiry(true));
   Optional<Instant> expiry = result.expiryTime();
   System.out.println("Expiry of found doc: " +
   expiry);

3  // Preserve expiry during KV replace
   collection.replace("my-document3", json,
      replaceOptions().preserveExpiry(true));

4  // Update the expiry field
   GetResult result = collection.getAndTouch("my-
   document3", Duration.ofDays(1));
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Java | SQL++ Queries

SQL++ queries is the interface for DDL and DML operations.

Couchbase SDK provides 2 approaches to parameterized SQL++ queries:

1. Example using Positional parameter

1. Example using Named parameter

```
1  // Parameterized query using positional parameter
   QueryResult result = cluster.query(
      "select count(*) from `travel-
   sample`.inventory.airline where country = ?",

   queryOptions().parameters(JsonArray.from("France")
   ));

   // Parameterized query using named parameter
2  QueryResult result = cluster.query(
      "select count(*) from `travel-
   sample`.inventory.airline where country =
   $country",

   queryOptions().parameters(JsonObject.create().put(
   "country", "France")));
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Transactions

Code examples in Java

Transaction Overview | Read Isolation

Couchbase transactions extends Read Committed Isolation level

Standard Read Committed

- Ensures that only committed data is visible to transactions.
- Does not guarantee a consistent view if data changes during the transaction.

Couchbase Monotonic Atomic View (MAV)

- Extends Standard Read Committed to **guarantee consistent snapshot of the database for the duration of the transaction**.
- All reads within a transaction reflect a monotonic view: the latest committed state at the time of the first read, regardless of concurrent changes.
- Prevents "non-monotonic reads," ensuring that data already read does not appear to roll back or change inconsistently

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Transaction Overview | Benefits of MAV

Atomicity and Isolation

- Combines the benefits of atomic writes with a consistent read view, providing a robust foundation for distributed systems.

Optimized for Scalability

- Ensures transactional consistency in a distributed environment while minimizing performance overhead compared to higher isolation levels like Serializable

Preventing Anomalies

- Protects against common anomalies like "write skew" and inconsistent reads within the same transaction.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Transaction Mechanics (ACID)

Transactions are initiated via a lambda and each run of lambda is called an attempt.

1. **Prepare**

Each transaction attempt adds an entry to a metadata document called an **Active Transaction Record (ATR)** in the Couchbase cluster.

2. **Mutate**

Document mutations don't directly alter the document body. Instead, changes are staged alongside the document, remaining invisible to Couchbase until the commit.

3. **Cleanup**

There are **safety mechanisms** to ensure that leftover staged changes from a failed transaction cannot block live transactions indefinitely.

4. **Commit**

The transaction is committed only after the lambda completes successfully, updating the ATR entry as an 'atomic commit' signal.

5. **Rollback**

If an exception occurs, the attempt is rolled back. Retries depend on the error type. Unretried transactions throw a `TransactionFailedException`.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Transactions | Create Transaction

Most SDK allows transactions in both Synchronous and Asynchronous APIs.

```
1   TransactionResult result =
    cluster.transactions().run((ctx) -> {
      // Transactional operations
    } catch
    (TransactionCommitAmbiguousException e) {
       throw logCommitAmbiguousError(e);
    } catch (TransactionFailedException e) {
       throw logFailure(e);
    }
```

1. Transaction requires application logic to be encapsulated within a lambda (`cluster.transactions()->run`)
   - Include any conditional logic: KV, SQL++, mutations
   - On lambda success, automatically committed.
   - If at any point an error occurs, the transaction will rollback and the lambda may run again.

TIPS

- Transactions operate on documents (not sub-document level)
- Avoid change operations directly in Transaction

  NOT **`collection.insert()`** → INSTEAD **`ctx.insert(collection,..)`**
- Read Couchbase transaction mechanics for SDK on our documentation website.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Transactions | Code Sample

```
try {
   TransactionResult result = cluster.transactions().run((ctx) -> {
       // Inserting a doc:
       ctx.insert(collection, "doc-a", JsonObject.create());

       // Getting documents:
       TransactionGetResult docA = ctx.get(collection, "doc-a");

       // Performing a SELECT SQL++ query against a scope:
       TransactionQueryResult qr = ctx.query(inventory, "SELECT * FROM hotel WHERE country = $1",
              TransactionQueryOptions.queryOptions().parameters(JsonArray.from("United Kingdom")));
       var rows = qr.rowsAs(JsonObject.class);

       // Performing an UPDATE SQL++ query on multiple documents, in the `inventory` scope:
       ctx.query(inventory, "UPDATE route SET airlineid = $1 WHERE airline = $2",
               TransactionQueryOptions.queryOptions().parameters(JsonArray.from("airline_137", "AF")));
   });
} catch (TransactionCommitAmbiguousException e) {
   throw logCommitAmbiguousError(e);
} catch (TransactionFailedException e) {
   throw logFailure(e);
}
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Asynchronous API

Code examples in Java

Async & Reactive API | Overview

What is Asynchronous API

- Designed for non-blocking operations, leveraging the reactive programming model to handle high-throughput use cases efficiently.
- Based on CompletableFuture (Java SDK) or similar constructs in other languages to handle asynchronous tasks.

Key Features

- **Non-Blocking I/O:** Enables efficient utilization of system resources.
- **Reactive Programming:** Provides a scalable way to manage multiple concurrent operations.
- **High Performance:** Ideal for latency-sensitive applications where blocking threads is a bottleneck

Common Use Cases

- **Real-time applications** requiring low-latency responses.
- **High-throughput systems**, such as web services or event-driven applications.
- **Microservices architectures** where scalability is crucial

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Async Operations Primer

The Scenario - User Browses Items on eCommerce Site

When the page loads, the application must:

1. **Fetch Product Metadata:** Price, description, and images.
2. **Fetch User Profile:** Loyalty Tier, preferences, and recent searches.
3. **Fetch Inventory Levels:** Real-time stock across multiple warehouses.
4. **Log an Analytics Event:** Record that the user viewed this specific item.

Why Async?

If you used **Synchronous (Blocking)** calls, the execution would look like this:

- Get Product (wait 2ms) $\rightarrow$ Get Profile (wait 2ms) $\rightarrow$ Get Inventory (wait 3ms) = **7ms total.**

With **Asynchronous** operations (using the Couchbase Java, Node.js, or .NET SDKs), you fire all requests simultaneously:

- Get Product
  Get Profile $\rightarrow$ **Total time: ~3ms** (the duration of the slowest single call).
  Get Inventory /

Async & Reactive API | Asynchronous vs Synchronous

	Asynchronous API	Synchronous API
Use Cases	• High concurrency, low latency applications.	• Simpler workflows or lower concurrency.
	• Need to handle thousands of parallel calls.	• Applications prioritizing ease of use.
	• Event-driven or reactive programming models.	• Suitable for quick prototyping or debugging.
Benefits	• Non-blocking, maximizes throughput.	• Straightforward implementation.
	• Prevents thread exhaustion.	• Easier to debug and maintain for simple cases.
Considerations	• Requires a reactive mindset for development.	• Blocks threads, potentially limiting scalability.
	• May involve steeper learning curves.	• Not ideal for high-concurrency environments.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Async & Reactive API | Asynchronous API support

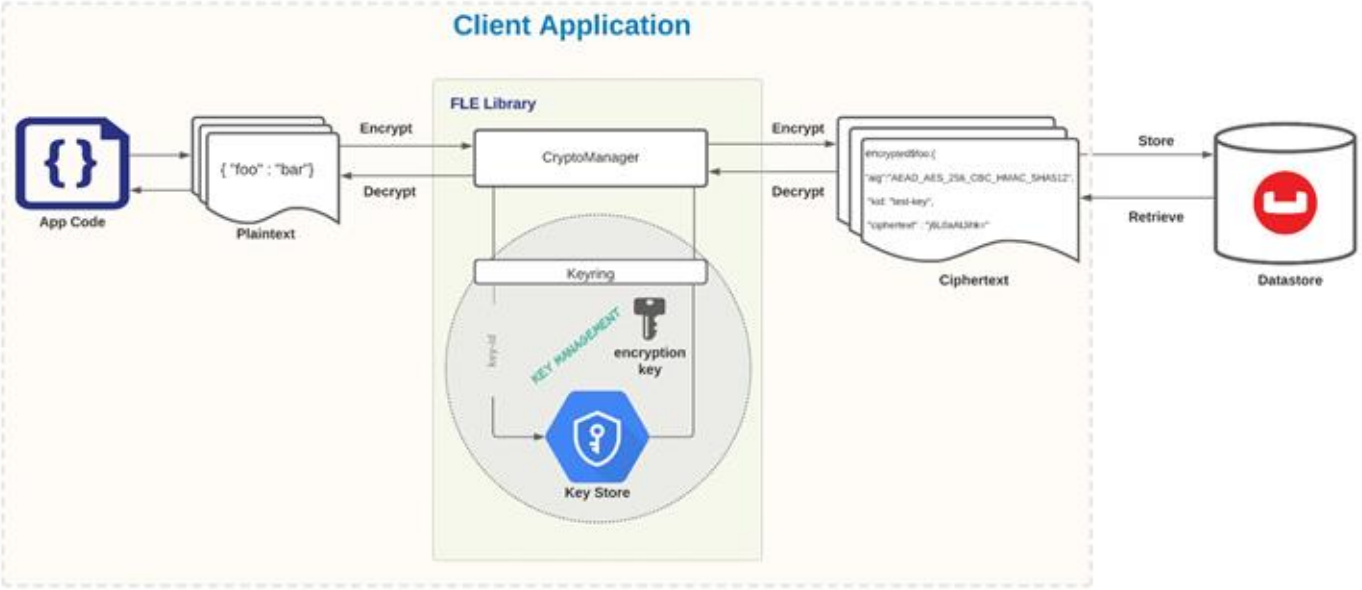
SDK	Asynchronous API		Synchronous API		Notes
Java	✓	Supported	✓	Supported	Async API based on CompletableFuture and reactive support with Project Reactor.
.NET	✓	Supported	✓	Supported	Async API via Task-based Asynchronous Pattern (TAP) and sync for simpler use cases.
Node.js	✓	Supported	✗	Not Supported	Fully asynchronous; aligns with Node.js event-driven architecture.
Python	✓	Supported	✓	Supported	Async API via Asyncio and sync API for simplicity.
Go	✓	Supported	✓	Supported	Async API only via Goroutines. Native Couchbase Go SDK only supports synchronous API.
C	✓	Supported	✗	Not Supported	Libcouchbase is an asynchronous client that may also be used in blocking mode.
Ruby	✗	Not Supported	✓	Supported	Simplified synchronous API for Ruby developers.
Scala	✓	Supported	✓	Supported	Allows the use, and mixing, of three distinct APIs: blocking, asynchronous, and reactive.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

SDK Field Level Encryption

Field Level Encryption Overview

- Fields within a JSON document can be securely encrypted by the SDK to support FIPS 140-2 compliance.
- This is a client-side implementation, with encryption and decryption handled by the Couchbase client SDK.



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Field Level Encryption Framework in SDKs

1. **Key** has an ID and a byte array with key material.
2. **Keyring** provides access to keys.
3. **Encrypter** encrypts values. It uses Keyring to get Key.
4. **Decrypter** decrypts values. It uses Keyring to get Key.

```
{
   "maxim": "The enemy knows the system"
}
```

Encrypted JSON

```
{
   "encrypted$maxim": {
       "alg": "AEAD_AES_256_CBC_HMAC_SHA512",
       "kid": "myKey",
       "ciphertext": "u+ssT1wJoDxk08hdZDf..."
   }
}
```

```
1 interface Key {
    String id();
    byte[] bytes();
  }
2 interface Keyring {
    Optional<Key> get(String keyId);
  }
3 interface Encrypter {
    EncryptionResult encrypt(byte[] plaintext);
  }
4 interface Decrypter {
    String algorithm();
    byte[] decrypt(EncryptionResult encrypted);
  }
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Integration Example | SpringVaultTransitProvider

Hashicorp Vault can be used as the Key Management Provider to securely encrypt and decrypt sensitive fields in JSON documents

1. **Define VaultEndpoint:** specifies the location of the Vault server
2. **Create VaultTemplate:** securely establishes the connection to the Vault server
3. **Initialize SpringVaultTransitProvider:** interfaces with the Vault Secrets Engine
4. **Configure a CryptoManager**: setup the decrypter and the encrypter components

```
1  VaultEndpoint endpoint = VaultEndpoint.create("127.0.0.1", 8200);
   endpoint.setScheme("https");

2  VaultTemplate vaultTemplate = new VaultTemplate(
       endpoint,
       new TokenAuthentication("<myAuthToken>")
   );

3  SpringVaultTransitProvider transit = new SpringVaultTransitProvider(
       vaultTemplate.opsForTransit()
   );

4  CryptoManager cryptoManager = DefaultCryptoManager.builder()
       .decrypter(transit.decrypter())
       .defaultEncrypter(transit.encrypterForKey("myKey"))
       .build();
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Error Handling

Error Handling | Request Lifecycle Overview

Pre-Dispatch Phase

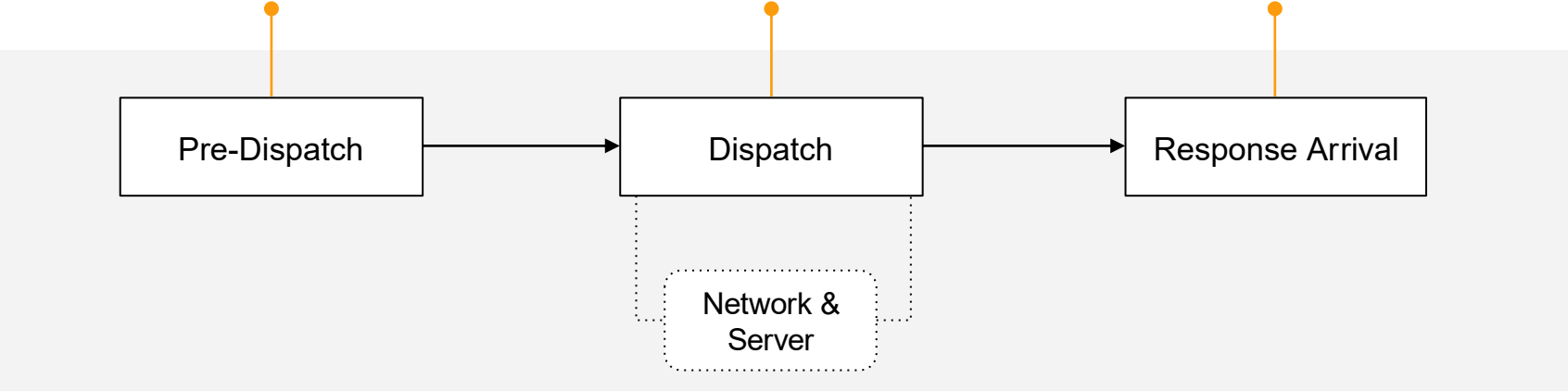
The SDK creates the request and identifies the appropriate socket or endpoint for operation dispatch.

Dispatch Phase

The operation is sent over the network, awaiting a server response. Retryability during this phase depends on the operation's idempotence.

Response Arrival Phase

Upon receiving the server's response, the SDK determines the next steps, aiming for successful operation completion.

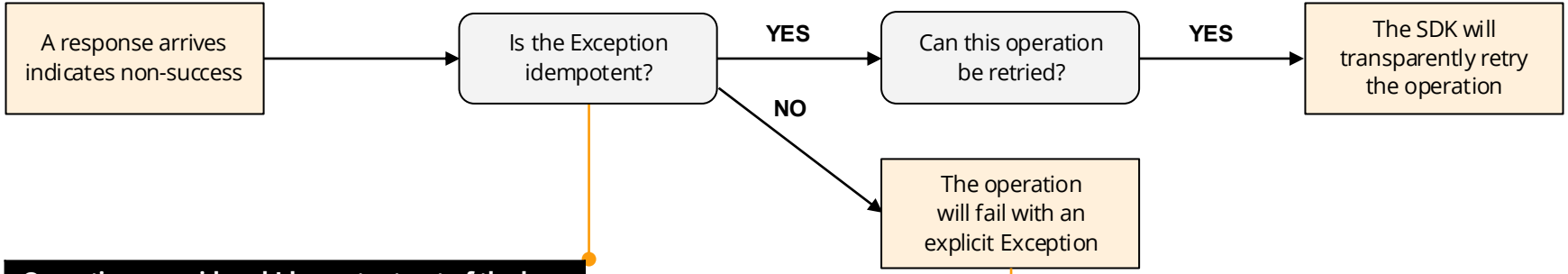


Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Error Handling | Approach

While all SDKs follow common approaches for Error handling, each language has its own nuances.

For example, in Java, depending on the state, relevant exceptions are raised.



Operations considered Idempotent out of the box	
`Cluster`	`search`, `ping`, `waitUntilReady`
`Bucket`	`view`, `ping`, `waitUntilReady`
`Collection`	`get`, `lookupIn`, `getAnyReplica`, `getAllReplicas`, `exists`
`Other`	Management commands that only retrieve information

- For example, if you perform an `insert` operation and the document already exists, the SDK will fail the operation with a `DocumentExistsException`.
- Failures in all other cases - especially during pre-dispatch and dispatch, will result in either a `TimeoutException` or a `RequestCanceledException`

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

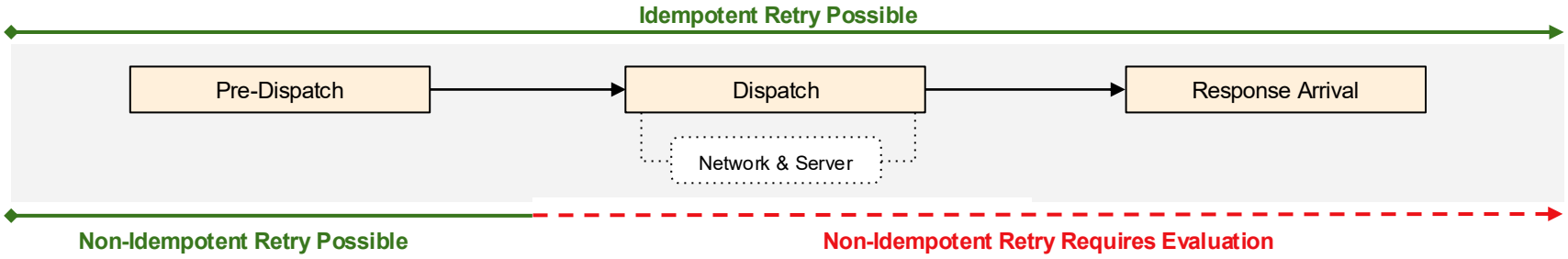
Error Handling | Idempotent vs. Non-Idempotent Requests

Idempotent Operations
Can be executed multiple times without changing the result beyond the initial application.

- **Examples**: get, exists, read-only queries
- **Retry Logic**: safe to retry upon failure, as repeated execution doesn't alter the final state
- **Exception Handling:** failures in idempotent operations may lead to automatic retries by the SDK.
- **Best Practices:** whenever possible, structure operations to be idempotent to facilitate safer retries

Non-Idempotent Operations
May produce different outcomes with repeated execution, potentially leading to unintended side effects.

- **Examples**: insert, replace, update operations
- **Retry Logic**: requires careful handling to avoid data inconsistencies, as retries may alter outcomes
- **Exception Handling:** failures often result in exceptions indicating the need for manual intervention
- **Best Practices:** handle exceptions to preserve data integrity and design effective retry strategies



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Error Handling | TimeoutException

Symptom Indicator

- Indicates that an operation exceeded the allotted time before completion.
- A TimeoutException signifies underlying issues and serves as a safeguard against prolonged thread blocking, allowing applications to handle delays gracefully.

Exception Types

- `UnambiguousTimeoutException`: indicates a definite timeout without side effects on the server.
- `AmbiguousTimeoutException`: suggests a timeout where the operation's outcome on the server is uncertain.

Key Considerations

- A timeout is a symptom - not the root cause, highlighting underlying issues (e.g. network latency, resource constraints.
- Utilize the `ErrorContext` provided with the exception to diagnose and address the underlying problem effectively. Accessible via the `context()` method on the exception.

Best Practices

- Investigate and resolve the root causes leading to timeouts to enhance application performance and reliability.
- Utilize `ErrorContext` for deeper insights and root causes.
- Handle the generic `TimeoutException` by implementing retry mechanisms, especially for idempotent operations.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Error Handling | TimeoutException Example

Example of an `ErrorContext` available as a method on the `TimeoutException` through the `context()` getter

```
Reason: TIMEOUT
{"cancelled":true,"completed":true,"coreId":"0x5b36b0db00000001","idempotent":true,"reason":"TIMEO
UT",
"requestId":22,"requestType":"GetRequest","retried":14,"retryReasons":["ENDPOINT_NOT_AVAILABLE"
],
"service":{"bucket":"travel-
sample","collection":"_default","documentId":"airline_10226","opaque":"0x24",
"scope":"_default","type":"kv"},"timeoutMs":2500,"timings":{"totalMicros":2509049}}

at com.couchbase.client.java.AsyncUtils.block(AsyncUtils.java:51)
//   ... (rest of stack omitted) ...
```

1. A `GetRequest` timed out after `2500` ms
2. The document in question had the ID `airline_10226` and we used the `travel-sample` bucket.
3. The request has been retried 14 times and the reason was always `ENDPOINT_NOT_AVAILABLE`.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Error Handling | Best Practices

Implement Robust Exception Handling

Catch and manage exceptions to maintain application stability

Leverage ErrorContext

Use the contextual information to diagnose and address issues effectively

Configure Appropriate Retry Strategies

Set retry policies based on operation idempotence and the nature of errors.

Monitor and Log Errors

Maintain comprehensive logs to facilitate troubleshooting and optimization.

***Goal*: Enhance application resilience and user experience through proactive error management.**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Observability

Observability Foundation and Philosophy

Logging
What happened?

- Event logs give immutable time-stamped records or discrete events over time.

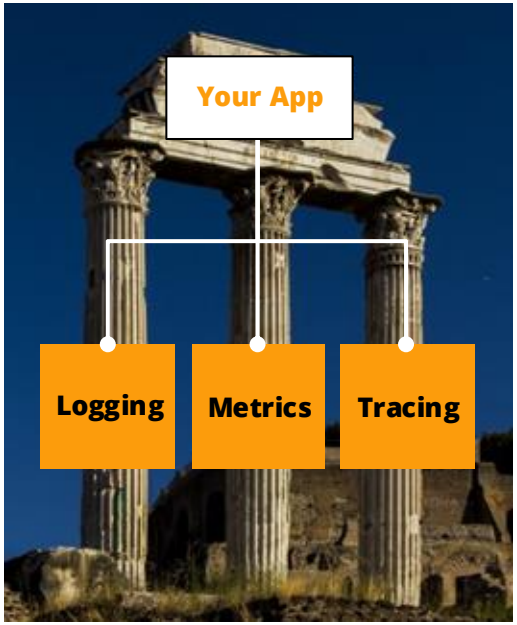
Metrics
How severe?

- Metrics are a numeric representation of data measured over intervals of time.

Tracing
Where to look?

- Tracing is a representation of causally related distributed events that document the end-to-end request of flow through a distributed system.

The three pillars



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Observability Foundation | Couchbase Implementation

Logging

Follow Platform Standards

- SDKs are part of a wider application and adopt common logging interfaces
- Logging is tagged to redact user data, with tools

Java	SLF4J
Golang	Allows integration
.NET	Logging providers, serilog

Metrics

Expose and Integrate

- Operation counts and latency histograms are all collected internally

OpenTelemetry	Current: Java, C, Scala, Kotlin
	Coming soon: Python, NodeJS, Go, C++, Ruby, PHP
Micrometer	Java, Scala, Kotlin
Prometheus	Java
LoggingMeter	Simple default passive logging available in Java and .NET

Tracing

Active and Passive

- Active tracing can be configured to OpenTelemetry

Active tracing to OpenTelemetry
Jaeger, OpenZipkin, Quarkus, Honeycomb.io, DataDog, etc.

- Passive tracing via Response Time Observability tracer

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Support for OpenTelemetry

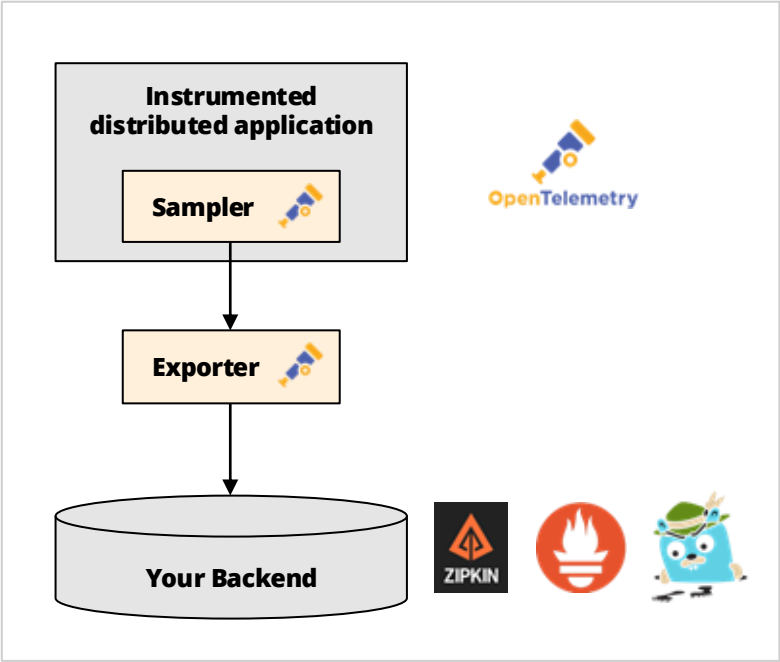
Strong Support for the Standard, Influenced Early OpenTracing and OTel (OpenTelemetry)

Overview

- OpenTelemetry is a vendor-agnostic, open standard designed to capture telemetry data, including metrics, logs, and traces.
- Couchbase SDKs are fully instrumented and documented to work seamlessly with OpenTelemetry.

Architecture

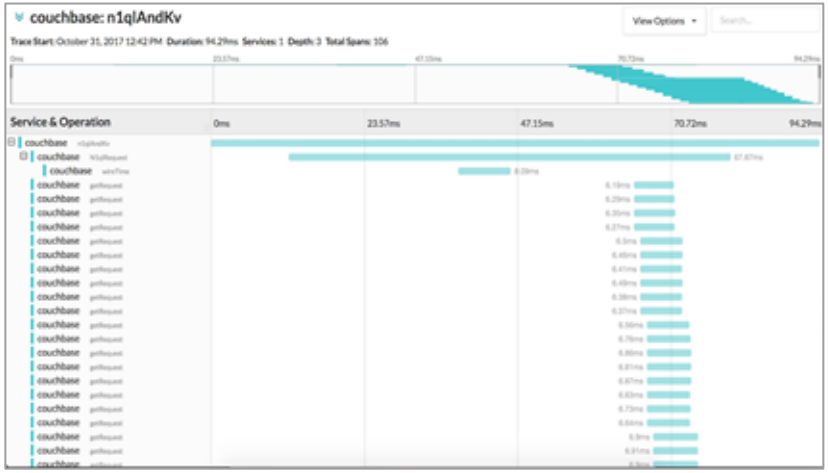
- **Sampler**: Decides what portion of your telemetry data to capture.
- **Exporter**: Handles the data transmission to your chosen backend.
- **Your Backend**: This can be any OpenTelemetry-compatible system like Jaeger, Zipkin, Prometheus, or cloud-based observability platforms such as Datadog or Honeycomb.io.

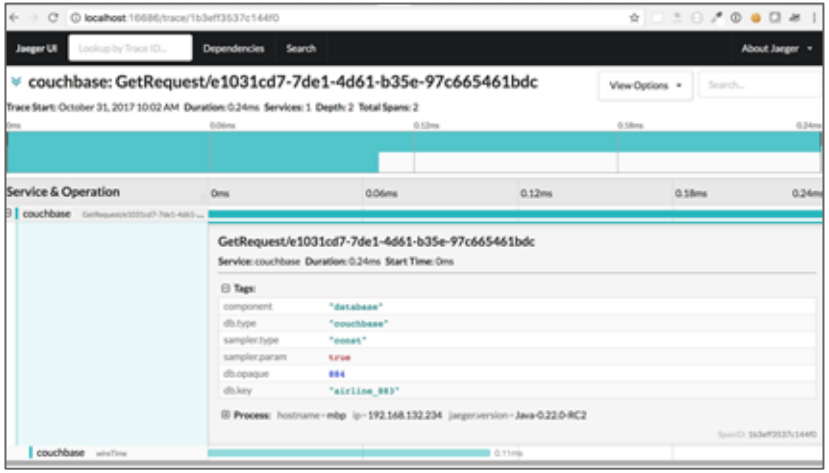


Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Pluggable Tracers

- Any OpenTracing tracer implementation supported
- Contextual OTel *Attributes* and *Baggage* are added to traces
- Drilling down from the application start point and get the overall picture





Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Thank you!

Couchbase